To initialize a GP object without any training data, we pass `None` as both the training features ( `train_x`) and labels (`train_y`). So, our prior GP is created as follows:

```
model = BaseGPModel(None, None, likelihood)
```

Finally, some bookkeeping is necessary before we can actually make predictions. First, we set the hyperparameters of the GP.

```
lengthscale = 1
noise = 1e-4

model.covar_module.lengthscale = lengthscale
model.likelihood.noise = noise

model.eval()
likelihood.eval()
```

We will save the discussion about what each of these hyperparameters controls for later in Chapter 3. For now, we will just use the values that I personally like to use as the default: 1 for the length scale and 0.0001 for the noise variance. The very last detail is to enable prediction mode in both the GP model and its likelihood by calling the `eval()` method from the corresponding objects.

With these bookkeeping tasks out of the way, we can now finally call this GP model on our test data to make predictions. We do this like so:

```
with torch.no_grad():
    predictive_distribution = likelihood(model(xs))
```

Remember in the `forward()` method of the model class, we return the multivariate Gaussian distribution, so that is the output when we pass some test data through our model using `model(xs)`. We also pass that same output through the likelihood function `likelihood`, which incorporates the noise variance into our predictions. In short, what we store in `predictive_distribution` is a multivariate Gaussian distribution that represents our prediction for the test points `xs`.

Moreover, we compute this object within a `torch.no_grad()` context, which is a good practice when we don't want PyTorch to keep track of the gradients of these computations.

**NOTE** **Disabling gradient when making predictions**

**We only want to compute the gradients of some operation when we'd like to optimize some parameters of our model using gradient descent. But when we want to make predictions, we should keep everything about our model fixed, so disabling gradient checking is appropriate.**

©Manning Publications Co. To comment go to liveBook
https://livebook.manning.com/#!/book/bayesian-optimization-in-action/discussion

**Licensed to ashwin sampath <ashwin.manning@gmail.com>**